

MODULE 2 · LEVEL 300

Requirements: Planning & Analysis with AI



From messy inputs to clear, testable, traceable requirements - using GitHub Copilot across issues, specs and Spaces

Day 1 · 75 minutes · Builds on Module 1 (the Copilot platform) · Novnex



Where we are, and today's shape

 approx 2 min

Module 1 gave you the platform. Modules 2-7 each take one SDLC phase. Today: Requirements.



Framing

Why requirements are high-stakes, and the map of where AI helps.

~12 min



Requirements with Copilot

Notes to stories to issues to specs - grounded, prioritised, traceable.

~47 min



Apply

By track, anti-patterns, the lab, and what's next.

~16 min

Everything today assumes Module 1's guardrails: ground the context, review everything, humans own the decisions.



Learning objectives

 approx 2 min

By the end of this module you will be able to:



Turn inputs into structure

Use Copilot to convert notes, transcripts and screenshots into clear requirements and user stories.



Write testable acceptance criteria

Draft Given/When/Then criteria and definition-of-done that QA and agents can act on.



Plan work AI can execute

Create issues, break down epics, and write specs that guide the Copilot agents.



Keep humans in control

Validate AI-drafted requirements with stakeholders and avoid invented scope.



Why requirements are the highest-leverage place to get AI right

 approx 3 min

The pain

- **Ambiguous inputs:**
 - notes, emails, transcripts, whiteboards - unstructured and contradictory.
- **Lost intent:**
 - the 'why' behind a request rarely survives to the backlog.
- **Untestable stories:**
 - 'make it faster' with no acceptance criteria.
- **Slow & manual:**
 - writing and grooming the backlog eats analyst time.

Why it matters

- A defect caught in requirements is far cheaper than one caught in production
- Agents amplify whatever you feed them - vague in, wrong out at scale
- Clear, testable requirements are the target every later phase aims at
- **So this is where AI leverage - and AI risk - is highest**



Where AI helps across requirements

 approx 4 min



Elicit & capture

Summarise notes; draft clarifying questions; read screenshots.



Analyse

Deduplicate, spot gaps, conflicts and ambiguity.



Structure

User stories, acceptance criteria, definition of done.



Plan

Issues, epics to sub-issues, specs for agents.



Prioritise & size

Draft MoSCoW/RICE and estimates - you decide.



Trace

Link requirement to issue to PR to test.

We'll take these in order. Throughout: Copilot drafts, the analyst and stakeholders decide.

PART 2

Requirements with Copilot

Concrete techniques: notes to stories to issues to specs - grounded, prioritised and traceable.

approx 47 min





From raw notes to structured requirements

⌚ approx 4 min

Paste messy notes into Copilot Chat and ask it to structure, deduplicate and flag ambiguity - then you verify.

Prompt to Copilot Chat

```
Here are raw notes from the stakeholder call.  
Extract discrete requirements. For each:  
  - a clear statement  
  - the stakeholder need behind it  
  - any ambiguity or open question  
Flag duplicates and conflicts. Group by theme.  
<paste notes>
```

What you get - and check

- **Structure:**
 - a clean list grouped by theme, with the 'why' kept.
- **Gaps surfaced:**
 - explicit open questions to take back to stakeholders.
- **Conflicts flagged:**
 - contradictions you might have missed.
- **Your job:**
 - confirm nothing was invented or dropped; resolve the questions.



User stories & testable acceptance criteria

⌚ approx 5 min

Copilot draft - story + Gherkin

Story:

As a returning customer, I want to reset my password so that I can regain access.

Acceptance criteria:

Given a valid account email

When I request a reset link

Then a single-use link is emailed within 1 min

And the link expires after 30 minutes

Why this matters

- **Testable:**
 - Given/When/Then becomes tests in Module 5 and a target for agents.
- **Complete:**
 - ask Copilot for edge cases and negative paths too.
- **Consistent:**
 - one format across the backlog; add your definition of done.
- **Verify:**
 - criteria must reflect real intent, not plausible-sounding filler.



Copilot in GitHub Issues

⌚ approx 4 min

Create and refine issues from a prompt - or a screenshot of a whiteboard - with title, body, labels and criteria filled in.

Create issue with Copilot

```
"Create an issue for password reset.  
Use the story and acceptance criteria  
we just wrote. Add labels: auth, ux."  
-> Issue #128 drafted:  
  title, body, acceptance criteria,  
  labels, suggested assignee
```

Handy in practice

- **From a screenshot:**
 - turn a whiteboard photo or mockup into a drafted issue.
- **Consistent issues:**
 - same shape every time; less blank-page friction.
- **Right where work lives:**
 - the issue is the unit agents and teams act on.
- **Still yours:**
 - edit before you save; you own the final wording.

The issue becomes the hand-off point: to a developer, or to the Copilot cloud agent (Module 1).



Break epics into workable pieces

⌚ approx 4 min

Ask Copilot to decompose a large requirement into a parent issue and right-sized sub-issues.

Prompt

```
Break epic #120 'Account self-service'  
into sub-issues, each independently  
shippable with its own acceptance  
criteria. Note dependencies.
```

```
-> #121 reset password    (no deps)  
    #122 change email     (needs #121)  
    #123 close account    (no deps)
```

Good decomposition

- **Right-sized:**
 - each piece shippable and independently testable.
- **Dependencies explicit:**
 - so sequencing and parallel work are clear.
- **Criteria per piece:**
 - every sub-issue carries its own acceptance criteria.
- **Review the cut:**
 - AI proposes a breakdown; you confirm it matches value and reality.



Spec-driven development: the spec as the contract

⌚ approx 5 min

For agent-built work, a written spec becomes the source of truth agents plan, build and test against. Toolkits like Spec Kit formalise this.

Spec-driven flow (Spec Kit)

```
/specify intent -> a structured spec
/plan    spec -> technical approach
/tasks   plan -> discrete tasks
         then agents implement
```

```
the spec stays the source of truth;
you refine intent, not just code
```

Why it matters here

- **Intent is the artefact:**
 - you engineer the spec; agents handle the code.
- **Reviewable:**
 - argue about the spec before any code is written.
- **Traceable:**
 - spec links to tasks, PRs and tests.
- **Requirements owner's lever:**
 - a great spec is how you steer the agents.



Ground requirements in Copilot Spaces

⌚ approx 4 min

Give Copilot your real product context so drafts use your terms, standards and prior decisions - not generic guesses.

Space: "Product Requirements"

```
repos:  acme/app, acme/product-docs
files:  glossary.md, personas.md,
        nfr-standards.md, past-ADRs/
instructions:
  Use our glossary and personas.
  Apply our NFR checklist.
  Cite the source doc for each point.
```

What it buys you

- **Your language:**
 - stories use your domain terms and personas.
- **Standards applied:**
 - non-functional checklist baked into drafts.
- **Consistency:**
 - whole team gets the same grounded answers.
- **Evergreen:**
 - GitHub sources auto-update as docs change.



Prioritisation - AI drafts, humans decide

⌚ approx 3 min

How Copilot helps

- **Draft a first cut:**
 - MoSCoW or RICE scores across the backlog.
- **Surface reasoning:**
 - ask why an item scored as it did.
- **Spot gaps:**
 - items with unclear value or missing data.
- **Re-sort fast:**
 - re-rank when a constraint changes.

Keep the human in charge

- AI has no access to your strategy, politics or budget
- Scores are inputs to a conversation, not a decision
- Value and effort estimates need real evidence
- Use it to prepare the trade-off discussion, not replace it



Estimation & risk - useful, with caveats

 approx 3 min



Where it helps

- Draft t-shirt sizes or story points to react to
- Surface risky or complex items early
- Suggest what could make an item smaller
- Turn a spec into a task list to size against



Handle with care

- AI can't know your team's velocity or context
- Estimates sound confident but aren't evidence
- Never commit a date on an AI number alone
- Calibrate against your own historical data



Traceability: keep requirements alive

⌚ approx 3 min

Requirements rot when they drift from the work. Copilot helps keep the thread from need to test.



- **Copilot helps:**
 - link issues to PRs, generate a coverage summary, and flag acceptance criteria with no matching test.
- **Ask it:**
 - "which criteria in #128 are not yet covered by a test?"



Working with stakeholders

 approx 3 min



Clarifying questions

Ask Copilot to generate the sharp questions your draft still leaves open.



Plain-language summaries

Turn a technical backlog into a stakeholder-friendly summary for sign-off.



Q&A over the spec

With a Space, stakeholders can ask 'is X in scope?' and get grounded answers.



Faster sign-off loops

Draft release notes and change summaries so approvals move quickly.



Live demo: from notes to ready-to-build

⌚ approx 5 min

Watch the thread form

- Messy notes -> structured requirements in chat
- One requirement -> user story + Given/When/Then
- Story -> a drafted GitHub Issue with criteria
- Epic -> sub-issues with dependencies
- Optional: delegate a well-scoped issue to the agent

Flow

1.
 - Paste notes; structure them.
2.
 - Draft a story + criteria.
3.
 - Create the issue from it.
4.
 - Break the epic down.
5.
 - Review; fix a weak criterion.



Responsible requirements with AI

🕒 approx 3 min



Invented scope

AI adds plausible requirements nobody asked for.

Guard: Trace every requirement to a real stakeholder need; challenge extras.



False precision

Specific numbers that were never agreed.

Guard: Confirm thresholds, SLAs and dates with stakeholders - don't ship guesses.



Lost intent / bias

The 'why' is dropped, or one voice dominates.

Guard: Keep the need with each item; check under-represented users are covered.



Rubber-stamping

Approving AI output without reading.

Guard: Human validation is the deliverable, not the draft - review every line.

PART 3

Apply it

By track, the anti-patterns to avoid, your lab, and where we go next.

approx 16 min





Requirements with Copilot - by track

 approx 4 min



Web / Drupal

Feature requests -> issues with criteria; scope a module; note migration and compatibility needs.



Data & Analytics

Define metrics precisely; capture source, grain and refresh; write data-quality acceptance criteria.



Power BI

Turn a report request into a spec: measures, filters, roles, and what 'correct' looks like.



RPA

Capture the current process into a clear spec: steps, exceptions, inputs, and the success condition.



Anti-patterns to avoid

 approx 3 min



Don't

- Generate a backlog and ship it unreviewed
- Accept vague criteria ('fast', 'user-friendly')
- Let AI set scope without stakeholder sign-off
- Bloat the spec because generating is cheap
- Skip the 'why' behind each requirement



Do

- Use AI for the draft; humans for the decision
- Insist on testable, specific criteria
- Ground drafts in a Space; cite sources
- Keep requirements small, traceable and current
- Validate with real stakeholders, every time



Your lab: requirements to build-ready

 approx 2 min

Lab 2 (this afternoon) - on your own track

- Start from a provided set of messy notes / a request for your track
- Use Copilot to structure requirements and write user stories + acceptance criteria
- Create a GitHub Issue and break one epic into sub-issues
- Ground your drafts in the provided requirements Space; validate and fix a weak criterion

Bring your questions - the facilitator and your group are there to help. Pair non-coders with developers.



Discussion & reflection

⌚ approx 4 min

Talk to the person next to you - three minutes:

- Where do requirements most often go wrong on your team - and which technique today would help most?
- What would you never want AI to decide in requirements - and how do you enforce that?

Ambiguity

Testability

Scope control

Stakeholder time

Traceability

We'll take a few answers before wrapping up.



Key takeaways

- Copilot turns messy inputs into structured, testable requirements - fast.
- Testable acceptance criteria are the contract for QA and for agents.
- Plan for agents: clear issues, good breakdowns, and specs as the source of truth.
- Ground everything in a Space; keep the thread from need to test.
- AI drafts; stakeholders and analysts decide. Validate - never rubber-stamp.

What's next

Module 3

Design

then

Lab 2 this afternoon

Questions?